

Least-Squares Based Aerodynamic Pressure Curve Matching

Explainable Cp-Curve Retrieval and Mapping for NACA 0012 Pressure Data

Python Written Assignment

Student: Tugce ADANAR

Course: Programming with Python

Institution: IU Internationale Hochschule GmbH

Submission Date: 30.06.2026

Declaration of Independent Work

This written assignment and the accompanying Python implementation were prepared independently for the course Programming with Python. The data-processing workflow, the curve-matching method, the result interpretation, and the written report reflect the author's own project work.

External sources were used for aerodynamic background, source-data references, and software documentation. The matching algorithm was implemented independently, and no third-party source code was copied into the submitted implementation.

All numerical results in this report are taken from the generated project files, especially results/best_matches.csv, results/error_audit.csv, results/mapping_results.csv, the processed input CSV files, and results/naca_cp_matching.sqlite.

Abstract

This report presents a Python workflow for matching pressure coefficient curves of the NACA 0012 airfoil. Four training Cp curves are compared with fifty reference Cp curves on a shared x/c grid. For each pair, the program calculates the sum of squared errors, the root mean squared error, and the maximum absolute Cp deviation. The lowest SSE defines the selected reference curve. To make the result traceable, the program also stores the Top-3 candidates and confidence margins.

The final run produced 200 curve comparisons, 4 best-match rows, 12 mapped test points, and 5 SQLite database tables. The smallest SSE occurred for training_2 matched with reference_27, with $SSE = 0.242730$ and $RMSE = 0.050816$. All 12 test points were accepted by the implemented mapping rule. The work stays within a numerical data-analysis scope: it compares existing Cp curves and does not claim CFD simulation, machine learning, inverse design, or prediction.

List of Contents

Declaration of Independent Work.....	2
Abstract	3
List of Figures	5
List of Tables	5
1 Introduction.....	6
2 Objectives.....	6
3 Technical Background	7
3.1 Pressure Coefficient and x/c	7
3.2 NACA 0012 Airfoil	7
3.3 Least-Squares Curve Matching.....	7
4 Experimental Data.....	7
4.1 Reference Curves from Gregory and O'Reilly.....	7
4.2 Training Curves from Ladson Pressure Data.....	9
4.3 Shared x/c Grid and Test Points	10
5 Methodology	11
5.1 Curve Representation.....	11
5.2 Point-wise Cp Difference	11
5.3 Sum of Squared Errors.....	11
5.4 Additional Error Metrics.....	11
5.5 Top-3 Ranking and Confidence Margin	12
5.6 Test Point Mapping Rule	12
5.7 Numerical Precision and Rounding	13

6 Software Design and Implementation.....	13
6.1 Program Architecture.....	13
6.2 Object-Oriented Structure.....	13
6.3 Input Validation and Exception Handling	14
6.4 Unit Testing	14
7 Computational Workflow	14
7.1 Reference Data Preparation	14
7.2 Training Data Preparation.....	14
7.3 Main Matching Run.....	14
8 Results.....	15
8.1 Generated Output Files	15
8.2 Best-Match Results.....	15
8.3 Top-3 Candidate Interpretation.....	15
8.4 Test Point Mapping Results.....	17
9 Discussion	17
9.1 Interpretation of the Matching Results	17
9.2 Strengths of the Method.....	17
9.3 Limitations.....	17
9.4 Why Machine Learning Was Not Used.....	18
9.5 Future Work.....	18
10 Reproducibility and Project Files.....	18
10.1 Run Commands	18
11 Conclusion	19
References.....	19
Appendix A: Important Code Snippets.....	20
A.1 SSE Calculation	20
A.2 Ranking Section.....	20
A.3 Mapping Threshold.....	20
Appendix B: SQLite Tables.....	20
Appendix C: Symbols and Abbreviations.....	20
Appendix D: Final Submission Checklist.....	21

List of Figures

Figure 1. Overall workflow of the Cp curve matching project. 8

Figure 2. Reference-data preprocessing from digitized Gregory and O'Reilly Cp curves. 8

Figure 3. Training-data preprocessing from Ladson pressure data..... 9

Figure 4. Bokeh visualization of training_3 with its Top-3 reference candidates and mapped test points. ... 16

List of Tables

Table 1. Main project scope decisions. 6

Table 2. Selected Ladson training cases. 9

Table 3. Experimental data sources and project roles..... 10

Table 4. Processed input data files..... 10

Table 5. Error metrics used in the matching algorithm..... 13

Table 6. Software modules and responsibilities..... 13

Table 7. Unit-test coverage. 14

Table 8. Generated result files. 15

Table 9. Best matching reference curves. 15

Table 10. Top-3 reference candidates for each training curve..... 15

Table 11. Test point mapping summary..... 17

Table 12. Important files used for reproducibility. 17

Table 13. SQLite database tables..... 20

Table 14. Symbols and abbreviations. 20

1 Introduction

Pressure coefficient curves are a practical way to describe how pressure varies along an airfoil surface. In this project, the chordwise position is written as x/c and the pressure coefficient as C_p . The use of x/c keeps the coordinate system normalized, so two C_p curves can be compared point by point on the same airfoil chord scale.

The chosen airfoil is NACA 0012. It is symmetric, well documented, and supported by experimental reference material. Gregory and O'Reilly reported low-speed measurements for NACA 0012, and Ladson published an additional experimental data set for the same airfoil section [1], [2]. These sources make the airfoil suitable for a small programming project focused on data preparation and curve comparison rather than geometry generation.

The project question is deliberately narrow: which reference C_p curve is closest to a given training C_p curve? The answer is obtained by calculating the least-squares error between curves sampled on the same x/c grid. The program also records RMSE, maximum local deviation, Top-3 ranking, confidence margins, test-point mapping results, SQLite output, and an interactive Bokeh visualization.

Table 1. Main project scope decisions.

Aspect	Decision	Reason
Airfoil	NACA 0012	The geometry is kept fixed so the work can focus on C_p -curve comparison.
Input data	Gregory and O'Reilly reference curves; Ladson training data	The program compares existing aerodynamic data and does not generate new flow data.
Curve scope	Upper-surface C_p curves	This keeps the preprocessing and matching scope consistent.
Grid	$x/c = 0.0100$ to 0.9400 , 94 points	SSE requires aligned C_p values at the same x/c locations.
Main method	Least-squares matching with SSE	The smallest SSE gives the closest numerical reference curve.
Excluded topics	ML, prediction, CFD, inverse design, optimization	These would change the assignment from curve matching into a different project.

Project file summary.

2 Objectives

The objective is to build a reproducible Python workflow that selects the closest NACA 0012 reference C_p curve for each training curve and saves enough information to check the decision afterwards.

1. Load processed training curves, reference curves, x/c grid values, and test points from CSV files.
2. Check that training and reference curves use the same x/c grid before comparing C_p values.
3. Compare 4 training C_p curves with 50 reference C_p curves using SSE.
4. Calculate RMSE and maximum absolute C_p deviation for every comparison.
5. Store the best match and the Top-3 ranked reference candidates for each training curve.
6. Map test C_p points to the selected reference curve using the implemented deviation threshold.
7. Save the results as CSV files and in an SQLite database.
8. Create a Bokeh HTML visualization and verify the mathematical parts with unit tests.

3 Technical Background

3.1 Pressure Coefficient and x/c

C_p is a non-dimensional pressure coefficient used in aerodynamic pressure-distribution data. In this report, C_p is treated as the dependent curve value, while x/c is the normalized position along the airfoil chord. This representation is enough for the implemented comparison because the program works with prepared C_p curves rather than raw pressure readings [3].

The project starts with existing C_p data. The preprocessing scripts clean, select, and interpolate the data so that the later SSE calculation can compare arrays of equal length.

3.2 NACA 0012 Airfoil

NACA 0012 is a symmetric four-digit NACA airfoil. For this assignment, its main advantage is not design complexity but availability: experimental data for this airfoil are well documented, and the shape is simple enough that the report can stay focused on Python, data handling, and numerical comparison [1]–[3].

3.3 Least-Squares Curve Matching

Least-squares matching compares two curves by taking the point-wise C_p difference, squaring each difference, and summing the squared values. Larger local deviations therefore influence the final error more strongly. After all reference candidates are checked, the candidate with the smallest SSE is selected as the best match.

4 Experimental Data

4.1 Reference Curves from Gregory and O'Reilly

The reference curves were prepared from digitized Gregory and O'Reilly pressure-coefficient curves for NACA 0012 [1]. The raw digitized files are WebPlotDigitizer CSV exports. The preprocessing script corrects the shifted x/c coordinates, removes invalid points, averages duplicate x/c entries, sorts the curves, and interpolates them onto the shared project grid.

The final reference set contains 50 reference curves and 94 x/c points per curve. The script uses 11 digitized alpha curves, with an alpha range from 0 to 16.5 deg. The common digitized x/c range used before writing the final grid is reported as 0.0009 to 0.9589 in the text; the full precision remains in the processed files.

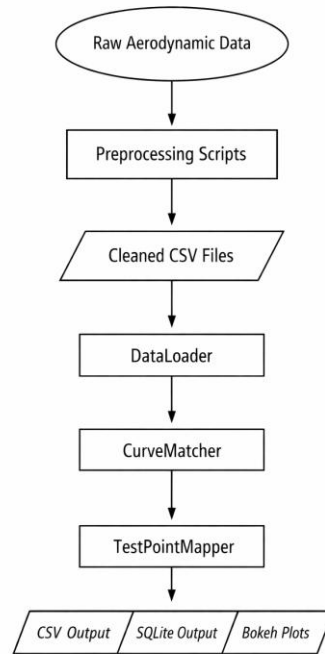


Figure 1. Overall workflow of the Cp curve matching project.

Source: Own illustration based on the implemented project workflow.

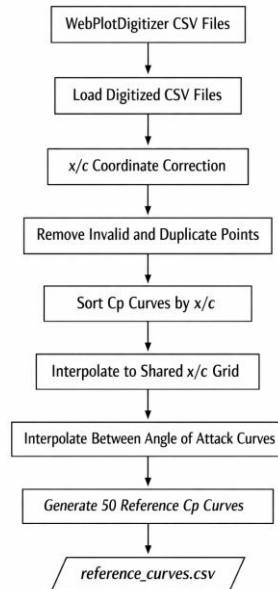


Figure 2. Reference-data preprocessing from digitized Gregory and O'Reilly Cp curves.

Source: Own illustration based on `prepare_digitized_cp_data.py`.

4.2 Training Curves from Ladson Pressure Data

The four training curves were generated from the Ladson pressure data [2]. The script reads CP_Ladson.dat, parses the zone headers, extracts the upper-surface branch, selects four representative cases, and interpolates them to the same x/c grid used for the reference library.

The selected cases cover a low-angle case, two higher-angle free-transition cases, and one fixed-transition high-Reynolds-number case. This gives the matcher several different Cp-curve shapes without expanding the assignment beyond one airfoil.

Table 2. Selected Ladson training cases.

Training curve	Reynolds number	Alpha	Transition
training 1	6 million	0.0169	free transition
training 2	6 million	10.0254	free transition
training 3	6 million	15.0299	free transition
training 4	9 million	14.9911	fixed transition

prepare_ladson_training_data.py output.

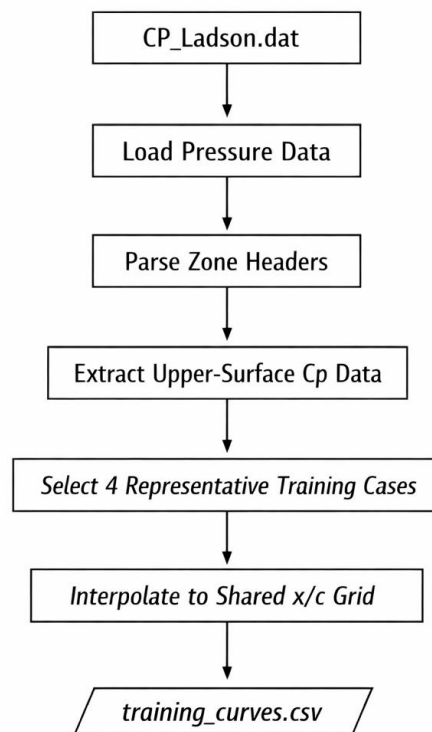


Figure 3. Training data preprocessing from Ladson pressure data.

Source: Own illustration based on *prepare_ladson_training_data.py*.

4.3 Shared x/c Grid and Test Points

The verified x/c grid is stored in data/x_grid.csv. It contains 94 points from 0.0100 to 0.9400. The same x column appears in training_curves.csv and reference_curves.csv, so the SSE calculation compares Cp values at matching chordwise positions.

The test-point file contains 12 points: three points for each training curve at $x/c = 0.1000$, 0.4000 , and 0.8000 . These points are used after the best reference curve has already been selected.

Table 3. Experimental data sources and project roles.

Data source	Role	Processing step	Output
Gregory & O'Reilly (1970)	Reference Cp curves	Digitization, x/c correction, interpolation, reference generation	data/reference_curves.csv
Ladson (1988)	Training Cp curves	DAT parsing, upper-surface extraction, case selection, interpolation	data/training_curves.csv
Project test points	Mapping check	Three points per training curve	data/test_points.csv

Project files and cited data sources.

Table 4. Processed input data files.

File	Verified shape	Important columns
data/reference_curves.csv	94 rows x 51 columns	x, reference_1, ..., reference_50
data/training_curves.csv	94 rows x 5 columns	x, training_1, training_2, training_3, training_4
data/test_points.csv	12 rows x 3 columns	x_test, cp_test, training_curve_id
data/x_grid.csv	94 rows x 1 column	x

Generated project data folder.

5 Methodology

5.1 Curve Representation

A Cp curve is stored as paired x/c and Cp arrays. With n points, one curve can be written as:

$$C = \{(x_i, C_{p,i})\}_{i=1}^n \quad (1)$$

In the code, these arrays are stored in Curve objects. TrainingCurve and ReferenceCurve inherit from Curve and therefore share the same basic validation rules.

5.2 Point-wise Cp Difference

For every training-reference pair, the first numerical step is the point-wise Cp difference:

$$\Delta C_{p,i} = C_{p,tr,i} - C_{p,ref,i} \quad (2)$$

The same difference vector is used for SSE, RMSE, and maximum absolute deviation.

5.3 Sum of Squared Errors

The main selection metric is the sum of squared errors. The selected reference is the one with the smallest total squared difference over the shared grid.

$$SSE = \sum_{i=1}^n (\Delta C_{p,i})^2 \quad (3)$$

$$best\ reference = reference\ with\ minimum\ SSE \quad (4)$$

5.4 Additional Error Metrics

SSE is useful for ranking, but it is not always intuitive by itself. RMSE gives an average error scale, and the maximum absolute deviation shows the worst local mismatch.

$$RMSE = \sqrt{\frac{SSE}{n}}$$

(5)

$$\max |\Delta C_p| = \max_i |\Delta C_{p,i}|$$

(6)

5.5 Top-3 Ranking and Confidence Margin

For each training curve, reference candidates are sorted by SSE. The first three rows are stored as the Top-3 candidates. The confidence margin is the difference between the Rank 2 SSE and Rank 1 SSE:

$$margin = SSE_{rank\ 2} - SSE_{rank\ 1}$$

(7)

$$relative\ margin = \frac{SSE_{rank\ 2} - SSE_{rank\ 1}}{SSE_{rank\ 1}}$$

(8)

The code also avoids a zero-division problem. If the best SSE is exactly zero, the relative margin is returned as 0.0 when both first and second SSE are zero; otherwise, it is returned as infinity.

5.6 Test Point Mapping Rule

After the best reference curve is known, each test point is checked against that reference curve. If the reference curve does not contain the exact test x/c value, linear interpolation provides the reference Cp value at that position.

$$\delta = |C_{p,test} - C_{(p,ref)(x_{test})}|$$

(9)

$$accepted\ if\ \delta \leq \max |\Delta C_p| \cdot \sqrt{2}$$

(10)

The factor $\sqrt{2}$ is used as a project-specific tolerance extension based on the maximum training deviation. It gives the mapping step a slightly wider acceptance range than the largest observed training-reference deviation, without changing the selected best reference curve. This threshold is used only inside this assignment and is not presented as a universal aerodynamic validation criterion.

5.7 Numerical Precision and Rounding

Numerical values are kept with full precision in the generated CSV files and in the SQLite database. Rounding is applied only in the written report to make the tables readable and consistent.

In the report, x/c values are shown with four decimal places when they appear as coordinates. Cp values and error metrics such as SSE, RMSE, maximum absolute Cp deviation, confidence margin, relative confidence margin, delta Cp, and acceptance limit are reported with six decimal places. Counts, curve IDs, file names, and qualitative descriptions are not rounded.

Table 5. Error metrics used in the matching algorithm.

Metric	Definition	Use in this project
SSE	sum of squared point-wise Cp differences	Main ranking criterion
RMSE	$\sqrt{\text{SSE} / n}$	Average Cp error scale
Max $ \Delta \text{Cp} $	largest absolute local Cp difference	Maximum training deviation for mapping
Top-3 rank	references sorted by increasing SSE	Shows nearby alternatives
confidence margin	$\text{SSE}_{\text{rank2}} - \text{SSE}_{\text{rank1}}$	Absolute separation between best and second-best
relative margin	$\text{margin} / \text{SSE}_{\text{rank1}}$	Separation relative to the best SSE

src/matcher.py and src/mapper.py.

6 Software Design and Implementation

6.1 Program Architecture

The code is organized as a small object-oriented pipeline. main.py starts the program. AssignmentApp then runs the steps in order: loading, matching, mapping, CSV export, SQLite storage, and Bokeh visualization. The design keeps file handling, mathematics, storage, and plotting in separate modules, which makes the project easier to evaluate and explain.

Table 6. Software modules and responsibilities.

Module/class	Responsibility	Reason for separation
main.py	Starts AssignmentApp	Keeps the entry point short
AssignmentApp	Runs the complete workflow	Controls the order of operations
DataLoader	Loads and validates CSV input	Keeps invalid data out of the matcher
Curve / TrainingCurve / ReferenceCurve	Store Cp curve data	Provides inheritance and shared validation
TestPoint	Stores one test Cp point	Keeps mapping input explicit
CurveMatcher	Calculates metrics, ranks candidates	Contains the mathematical decision logic
TestPointMapper	Checks mapped points	Applies the threshold after matching
DatabaseManager	Writes SQLite tables	Stores results persistently
Visualizer	Creates Bokeh HTML output	Supports visual inspection

Implemented source modules.

6.2 Object-Oriented Structure

The inheritance structure is intentionally small. Curve is the base class. TrainingCurve and ReferenceCurve inherit from it, so both complete curve types share the same storage and validation logic. TestPoint remains separate because it represents one measured point rather than a full Cp curve.

6.3 Input Validation and Exception Handling

Validation is used before calculations are made. Curve objects reject empty arrays, non-one-dimensional arrays, mismatched x and Cp lengths, and non-finite values. DataLoader checks missing files, empty CSV files, required columns, numeric columns, and x/c grid consistency. CurveMatcher repeats the x/c consistency check before comparing any training-reference pair.

The database step also has explicit error handling. If SQLite writing fails, DatabaseManager raises a RuntimeError that includes the database path and keeps the original SQLite error attached. This makes failure cases easier to locate than a generic crash message.

6.4 Unit Testing

The test module checks the mathematical metrics, ranking logic, x/c-grid consistency, input validation, and the test-point mapping rule. The verified test command, `python -m unittest discover tests`, reported 15 tests passed.

Table 7. Unit-test coverage.

Test area	Checked behavior	Result
Metrics	SSE, RMSE, and max absolute deviation on known arrays	Expected values are returned
Confidence margins	Best-vs-second SSE difference and relative margin	Calculated correctly
Ranking	Best match, second-best match, and Top-3 order	Smallest SSE receives Rank 1
Invalid input	Empty, mismatched, NaN, and infinity arrays	ValueError is raised
Grid checks	x_grid.csv, training_curves.csv, reference_curves.csv	Same x/c grid is confirmed
Mapping rule	Point at threshold and point above threshold	Boundary point accepted; above-limit point rejected
Final run	<code>python -m unittest discover tests</code>	15 tests passed

tests/test_metrics.py and verified test run.

7 Computational Workflow

7.1 Reference Data Preparation

The reference preprocessing step reads 11 digitized Gregory and O'Reilly alpha curves. After the x/c correction and cleaning stage, the curves are interpolated to the shared grid. Interpolation across alpha values then produces the 50 reference curves used by the matcher.

7.2 Training Data Preparation

The training preprocessing step reads the Ladson DAT file, extracts upper-surface pressure distributions, selects four training cases, and writes training_curves.csv. The written x/c range is 0.0100 to 0.9400 with 94 points.

7.3 Main Matching Run

The main run loads the processed files, checks the grids, compares every training-reference pair, saves error_audit.csv and best_matches.csv, maps 12 test points, writes the SQLite database, and creates cp_matching_plot.html. The verified command `python main.py` completed successfully and produced all expected result files.

8 Results

8.1 Generated Output Files

The final workflow creates CSV files for quick inspection, an SQLite database for structured storage, and a Bokeh HTML file for visual review. The error audit contains every comparison, while the best-match and mapping files summarize the selected results.

Table 8. Generated result files.

Output file	Verified content	Use
results/error_audit.csv	200 rows: 4 training curves x 50 reference curves	Full comparison audit
results/best_matches.csv	4 rows	Main best-match table
results/mapping_results.csv	12 rows	Mapped test-point decisions
results/naca_cp_matching.sqlite	5 database tables	Persistent result storage
results/cp_matching_plot.html	4 Bokeh tabs	Visual inspection of matches and mapped points

Verified output folder.

8.2 Best-Match Results

Table 9 lists the selected reference curve for each training curve. The strongest numerical match is training_2 to reference_27. The smallest relative margin occurs for training_3, where Rank 1 and Rank 2 are close compared with the other cases.

Table 9. Best matching reference curves.

Training	Best ref	SSE	RMSE	Max dev.	Second ref	Margin	Rel. margin
training_1	reference_1	0.321284	0.058463	0.300862	reference_2	0.352758	1.097960
training_2	reference_27	0.242730	0.050816	0.314592	reference_26	0.122231	0.503567
training_3	reference_41	0.553025	0.076702	0.350635	reference_40	0.042563	0.076965
training_4	reference_42	0.415964	0.066522	0.328749	reference_43	0.110072	0.264619

results/best_matches.csv.

8.3 Top-3 Candidate Interpretation

The Top-3 candidates provide a short audit trail. They show whether the selected reference is clearly separated or whether neighboring references produce similar error values.

Table 10. Top-3 reference candidates for each training curve.

Training	Rank	Reference	SSE	RMSE	Max delta Cp
training_1	1	reference_1	0.321284	0.058463	0.300862
training_1	2	reference_2	0.674043	0.084680	0.395915
training_1	3	reference_3	1.166500	0.111398	0.490968
training_2	1	reference_27	0.242730	0.050816	0.314592
training_2	2	reference_26	0.364960	0.062310	0.302041
training_2	3	reference_28	0.401168	0.065328	0.493776
training_3	1	reference_41	0.553025	0.076702	0.350635
training_3	2	reference_40	0.595589	0.079599	0.510236
training_3	3	reference_42	0.762853	0.090086	0.237655
training_4	1	reference_42	0.415964	0.066522	0.328749
training_4	2	reference_43	0.526036	0.074807	0.181234
training_4	3	reference_41	0.538111	0.075661	0.488350

results/error_audit.csv.

For training_1, reference_1 is well separated from reference_2, and the relative margin is larger than 1.0. For training_2, reference_27 is the best candidate, but reference_26 remains a nearby alternative. The

closest case is training_3: reference_41 is selected, while reference_40 has a similar SSE. For training_4, reference_42 is selected and reference_43 is the closest alternative.

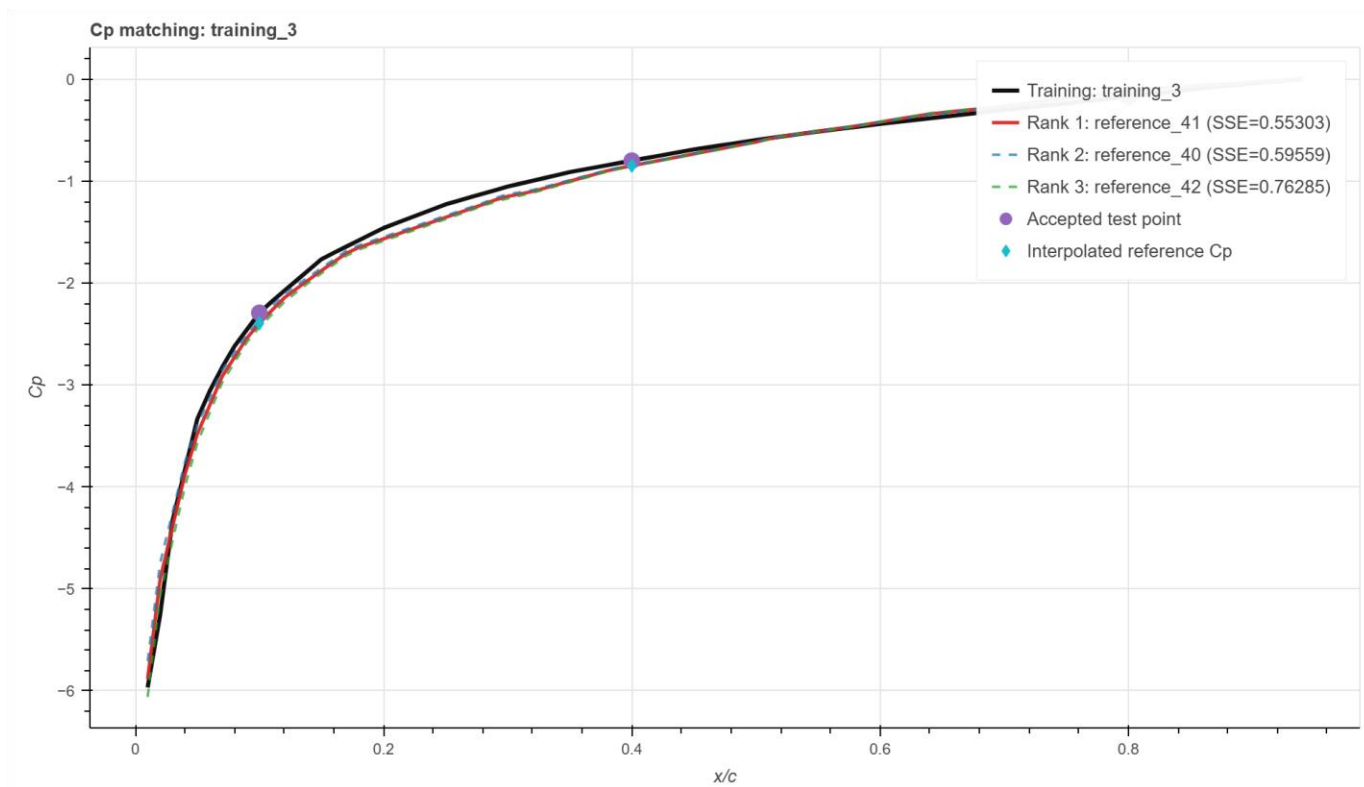


Figure 4. Bokeh visualization of training_3 with its Top-3 reference candidates and mapped test points.

Source: Own Bokeh output generated by `results/cp_matching_plot.html`. The interactive Bokeh HTML output supports hover tooltips for inspecting x/c and C_p values and legend toggling for hiding or showing individual curves.

8.4 Test Point Mapping Results

The mapping step evaluates three test points for each training curve. All points are checked against the best reference curve selected for their training curve. In the final output, all 12 points are accepted.

Table 11. Test point mapping summary.

Training	Points	Accepted	Rejected	Largest delta	Limit	Comment
training_1	3	3	0	0.082912	0.425483	All accepted; largest delta below limit
training_2	3	3	0	0.038879	0.444901	All accepted; largest delta below limit
training_3	3	3	0	0.102882	0.495873	All accepted; largest delta below limit
training_4	3	3	0	0.070835	0.464921	All accepted; largest delta below limit

results/mapping_results.csv.

9 Discussion

9.1 Interpretation of the Matching Results

The best match results follow the expected logic of the project: each training curve is assigned to the reference curve with the lowest SSE. The result for training_2 is the cleanest numerical match in terms of SSE. The match for training_3 needs more careful interpretation because the margin between the first and second candidate is small. This does not make the result wrong; it simply means that the two closest reference curves are similar on the shared x/c grid.

The mapping results are also consistent with the matching output. The largest test-point deviation is 0.102882 for training_3, which is still below its acceptance limit of 0.495873. Therefore, all mapped points remain inside the implemented threshold.

9.2 Strengths of the Method

- SSE-based selection is simple to follow: the smallest SSE gives the selected reference curve.
- The full audit table makes the ranking reproducible; the result does not depend on hidden decisions.
- The Top-3 output is especially useful for training_3, where the closest candidates are not far apart.
- Using one shared x/c grid avoids comparing Cp values at different chordwise positions.
- The SQLite database keeps inputs and outputs available after the program runs.

9.3 Limitations

- The Gregory and O'Reilly reference curves were digitized from a figure, so small manual digitization errors are possible.
- Some reference curves are generated by interpolation between digitized alpha cases. They are useful candidates, but they are not independent wind-tunnel measurements at every intermediate condition.

- Only upper-surface Cp curves are used. This keeps the project focused but excludes lower-surface pressure behavior.
- The method compares Cp values only. It does not model the flow physics, boundary layer effects, stall behavior, or lift generation directly.
- The mapping threshold is specific to this project and should not be interpreted as a universal aerodynamic acceptance rule.
- Because SSE squares the differences, a large local deviation can influence the ranking strongly. A later robustness check could compare the ranking with MAE.

9.4 Why Machine Learning Was Not Used

Machine learning was not needed for the question addressed here. The task is to retrieve the closest curve from a known reference set. A direct least-squares comparison is simpler, deterministic, and easier to verify with unit tests. For this assignment, this is a better fit than training a predictive model.

9.5 Future Work

Several extensions would be useful, but they are outside the submitted implementation. The most direct extension would be to include lower-surface Cp curves and check whether the selected references stay consistent. A weighted SSE could also be evaluated, especially near the leading edge where pressure gradients may be stronger. Other additions are an MAE comparison, uncertainty bands for digitized data, and a small user interface for inspecting candidate curves interactively.

10 Reproducibility and Project Files

The project is reproducible from the processed data files and the included source code. Raw data, processed input files, source modules, tests, and generated results are kept in separate folders.

Table 12. Important files used for reproducibility.

Category	Files
Main scripts	main.py; prepare_digitized_cp_data.py; prepare_ladson_training_data.py
Source modules	src/app.py; src/data_loader.py; src/curves.py; src/matcher.py; src/mapper.py; src/database.py; src/visualizer.py
Tests	tests/test_metrics.py
Processed input data	data/x_grid.csv; data/reference_curves.csv; data/training_curves.csv; data/test_points.csv
Results	results/error_audit.csv; results/best_matches.csv; results/mapping_results.csv; results/naca_cp_matching.sqlite; results/cp_matching_plot.html
Raw/reference material	data/raw/gregory_oreilly_1970.pdf; data/raw/source_curves/*.csv; data/raw/training_data/CP_Ladson.dat
Documentation	README.md; data/raw/digitization_protocoll.txt; data/raw/Decision_List.docx

Final project folder.

10.1 Run Commands

The workflow was checked with the following commands:

```
python -m pip install -r requirements.txt
python prepare_digitized_cp_data.py
python prepare_ladson_training_data.py
python main.py
python -m unittest discover tests
```

The main workflow ran successfully and produced the expected output files. Running the unit tests with `python -m unittest discover tests` gave 15 passing tests. The project was executed with Python 3.13.9. All required third-party packages are listed in requirements.txt, allowing the project to be reproduced in a compatible Python environment.

11 Conclusion

This project produced a complete least-squares workflow for matching NACA 0012 pressure coefficient curves. Four Ladson-derived training curves were compared with fifty Gregory-and-O'Reilly-derived reference curves on a shared x/c grid from 0.0100 to 0.9400. The final run generated 200 audited curve comparisons and selected one best reference curve for each training curve.

The selected matches were training_1 -> reference_1, training_2 -> reference_27, training_3 -> reference_41, and training_4 -> reference_42. The strongest match was training_2 -> reference_27 with SSE = 0.242730 and RMSE = 0.050816. The least separated case was training_3, where the relative confidence margin was 0.076965. This result shows why the Top-3 ranking is useful: it identifies cases where the best candidate is only slightly better than the next alternative.

The mapping step evaluated 12 test C_p points. All 12 were accepted by the implemented threshold, and the largest delta C_p value remained below the corresponding acceptance limit. Overall, the project demonstrates a compact but traceable aerodynamic data-analysis pipeline. Its contribution is not only the SSE formula, but the surrounding workflow: preprocessing, validation, object-oriented curve representation, ranking, mapping, SQLite storage, visualization, and unit testing.

References

- [1] N. Gregory and C. L. O'Reilly, Low-Speed Aerodynamic Characteristics of NACA 0012 Aerofoil Section, Including the Effects of Upper-Surface Roughness Simulating Hoar Frost, A.R.C. R&M No. 3726, 1970. Available: <https://reports.aerade.cranfield.ac.uk/handle/1826.2/3003>. Accessed: Jun. 30, 2026.
- [2] C. L. Ladson, Effects of Independent Variation of Mach and Reynolds Numbers on the Low-Speed Aerodynamic Characteristics of the NACA 0012 Airfoil Section, NASA Technical Memorandum 4074, 1988. Available: <https://ntrs.nasa.gov/citations/19880019495>. Accessed: Jun. 30, 2026.
- [3] I. H. Abbott and A. E. von Doenhoff, Theory of Wing Sections, Including a Summary of Airfoil Data. New York, NY, USA: McGraw-Hill, 1949.
- [4] A. Rohatgi, WebPlotDigitizer. Automeris. Available: <https://automeris.io>. Accessed: Jun. 27, 2026.
- [5] NumPy Developers, NumPy Documentation. Available: <https://numpy.org/doc/stable/>. Accessed: Jun. 27, 2026.
- [6] pandas Developers, pandas Documentation. Available: <https://pandas.pydata.org/docs/>. Accessed: Jun. 27, 2026.
- [7] Bokeh Development Team, Bokeh Documentation. Available: <https://docs.bokeh.org/en/latest/>. Accessed: Jun. 27, 2026.
- [8] Python Software Foundation, sqlite3 - DB-API 2.0 interface for SQLite databases. Available: <https://docs.python.org/3/library/sqlite3.html>. Accessed: Jun. 27, 2026.
- [9] Python Software Foundation, unittest - Unit testing framework. Available: <https://docs.python.org/3/library/unittest.html>. Accessed: Jun. 27, 2026.

Appendix A: Important Code Snippets

Only the central snippets are included here. The complete source files remain in the project folder.

A.1 SSE Calculation

```
@classmethod
def calculate_sse(cls, training_cp: np.ndarray, reference_cp: np.ndarray) -> float:
    """Calculate SSE, the main value used to choose the closest curve."""
    difference = cls._validated_difference(training_cp, reference_cp)
    return float(np.sum(np.square(difference)))
```

The snippet validates the two Cp arrays, calculates their point-wise difference, squares the differences, and sums them. It implements the SSE calculation used for ranking.

A.2 Ranking Section

```
audit = pd.DataFrame(rows)
audit = audit.sort_values(
    ["training_curve_id", "sse", "reference_curve_id"],
    kind="stable",
).reset_index(drop=True)
audit["rank"] = audit.groupby("training_curve_id").cumcount() + 1
audit["is_best_match"] = audit["rank"].eq(1)
top_three = audit.loc[audit["rank"] <= 3].copy().reset_index(drop=True)
```

This code sorts every reference candidate by SSE for each training curve. Rank 1 is the selected best match, and ranks 1 to 3 form the Top-3 table.

A.3 Mapping Threshold

```
reference_cp = self._interpolate_reference(references[reference_id], point.x_test)
delta_cp = abs(point.cp_test - reference_cp)
max_training_deviation = float(match["max_training_deviation"])
acceptance_limit = max_training_deviation * math.sqrt(2.0)
accepted = bool(delta_cp <= acceptance_limit)
```

This code applies the project-specific mapping rule. The reference Cp is interpolated at x_test, delta Cp is calculated, and the point is accepted only if the deviation is inside the threshold.

Appendix B: SQLite Tables

Table 12. SQLite database tables.

Table	Rows	Content
training_curves	376	Training Cp curves in long format: curve_id, x, cp
reference_curves	4700	Reference Cp curves in long format: curve_id, x, cp
error_audit	200	All training-reference comparisons with SSE, RMSE, max_delta_cp, rank, and is_best_match
best_matches	4	Best reference per training curve with confidence information
mapping_results	12	Mapped test points with interpolated reference Cp, delta Cp, limit, and accepted flag

results/naca_cp_matching.sqlite.

Appendix C: Symbols and Abbreviations

This appendix summarizes the main symbols, abbreviations, and project-specific terms used in the report.

Table 13. Symbols and abbreviations.

Symbol / abbreviation	Meaning	Use in this project
Cp	Pressure coefficient	Main dependent value of the airfoil pressure curves.

x/c	Normalized chordwise position	Shared coordinate used to compare Cp values point by point.
NACA 0012	Symmetric NACA airfoil section	Fixed airfoil geometry used for the curve-matching study.
SSE	Sum of squared errors	Main criterion for selecting the closest reference curve.
RMSE	Root mean squared error	Average error scale derived from SSE.
ΔC_p	Point-wise Cp difference	Difference between training and reference Cp values.
max $ \Delta C_p $	Maximum absolute Cp deviation	Largest local difference used as maximum training deviation for mapping.
n	Number of grid points	Total number of shared x/c positions in one curve.
i	Grid-point index	Index for one x/c position.
j	Reference-curve index	Index used when ranking reference candidates.
Training curve	Target Cp curve	Curve compared against the reference library.
Reference curve	Candidate Cp curve	Possible match selected by the smallest SSE.
Test point	Single x/c and Cp pair	Point checked against the selected reference curve.
Top-3 ranking	Three lowest-SSE references	Short audit trail for the best match.
Confidence margin	$SSE_rank2 - SSE_rank1$	Absolute separation between best and second-best reference.
Relative confidence margin	Margin divided by SSE_rank1	Scale-independent comparison of match separation.
SQLite	Lightweight database format	Stores curves, error audit, best matches, and mapping results.
CSV	Comma-separated values file	Input and output table format used by the workflow.
Bokeh	Python visualization library	Creates the interactive HTML plot for visual inspection.

Appendix D: Final Submission Checklist

- Draft notes and placeholder instructions have been removed from the report text.
- Numerical result tables are filled with values from the final CSV files and SQLite database.
- The report does not claim machine learning, prediction, CFD simulation, inverse design, or aircraft design.
- Gregory and O'Reilly data are used for reference curves; Ladson data are used for training curves.
- The mapping threshold is described as project-specific.
- The code runs with `python main.py`.
- The unit tests pass with `python -m unittest discover tests`
- References follow one IEEE-style numbering system.
- Only short code snippets are included in the appendix.
- No third-party source code is presented as original work.